CatanIA — Dual Repository Website Architecture & Go-Live Plan

A detailed implementation document for splitting the current CatanIA project into two Git repositories: one repository for AI training/local play and one repository for the public website/platform.

Primary objective: preserve the current CatanIA engine and AI-training workflow while building a separate, scalable web platform where users can create accounts, submit trained model weights, play against AIs, view rankings, watch Arena matches and eventually monetize their AIs.

Source basis. The current repository README says the engine is dependency-free, the browser is an interface over the authoritative server-side rules, training is the only package importing PyTorch, and the project currently has a stdlib web interface. The separate Website Structure document defines the intended public pages and product features.

1. Executive Decision

Use two repositories with a deliberately narrow interface between them.

```
REPOSITORY 1 — CatanIA
Purpose: game engine, AI development, training and local play

REPOSITORY 2 — CatanIA-Web
Purpose: public website, accounts, API, database, model registry,
uploads, online games, leaderboard, Arena orchestration,
tutorials, payments and administration
```

The web repository must **not** become a second implementation of Catan. It should consume a versioned, production-safe package/interface from the AI repository. The current README explicitly establishes one source of truth for rules, observation, action legality and server-side game state.

The critical boundary is therefore: **Git repository boundary ≠ game-logic boundary.** The game engine remains owned by CatanIA; the web repository owns everything necessary to expose it safely as a service.

2. Target Repository Structure

```
GitHub
■
■■■ TheoLindqvist4/CatanIA
■ ■■■ catan/ # authoritative game engine
■ ■■■ training/ # PPO / AlphaZero / self-play
■ ■■■ interfaces/ # CLI + local browser interface
■ ■■■ benchmark/
■ ■■■ configs/
■ ■■■ docs/
■ ■■■ tests/
■ ■■■ models/ # official local champions
■
■■■ TheoLindqvist4/CatanIA-Web
■■■ frontend/
■■■ backend/
■■■ migrations/
■■■ workers/
■■■ infra/
■■■ tests/
■■■ docs/
■■■ docker-compose.yml
■■■ README.md
```

The first repository remains usable by someone who clones it and wants to train an AI or play locally. The second repository is what you deploy to the Internet.

3. What Stays in CatanIA

Do not move the following into the web repository:

• catan/rules.py, state.py, topology.py, board.py, view.py, encoder.py and action_space.py.

• The baseline agents and game environment.

• PPO and AlphaZero training code.

• MCTS/determinization and self-play machinery.

• Training checkpoints and local training workflows.

• Benchmarking and research dashboards that are specifically for model development.

• The local CLI and local browser interface.

• The current official champion model files, subject to your existing promotion policy.

The README describes the current repository as having the engine in catan/, interfaces as the display layer, and training as the only package importing PyTorch. It also documents the 2,503-float observation, 325 discrete actions, hidden-information protections and the current champion/promotion system.

4. What Moves to CatanIA-Web

• Public frontend and all website pages.

• Authentication and account management.

• User profiles.

• AI profiles and AI version registry.

• Secure model-weight upload and validation.

• Online game rooms and matchmaking.

• Leaderboard and ELO presentation.

• Arena scheduling/orchestration.

• Arena TV live/replay presentation.

• Tutorial/content management.

• Subscriptions, payments and creator revenue accounting.

• Admin/moderation interfaces.

• Production database and migrations.

• Observability, deployment and infrastructure configuration.

5. The Boundary Between the Two Repositories

The most important engineering task before creating the web application is defining a stable **Model + Game Runtime Contract**.

```
CatanIA
■
■ versioned runtime/model contract
▼
CatanIA-Web

Contract must identify at minimum:
- ruleset version
- engine version
- observation version / size
- action-space version / size
- network architecture version
- inference mode
- model serialization format
- model checksum
- expected Python / PyTorch compatibility
```

The current project has already experienced changes where identical weights produced different results under different rules, and the README explicitly warns that win rates against the heuristic are only comparable within a rules version. This is a strong reason to make ruleset and model compatibility explicit in the web platform.

6. Weight-Only User Submission — Required Design

Users should submit **weights only**. They should never upload executable AI code to the production platform.

```
User
■
■■■ weights file
■
▼
Upload quarantine
■
▼
Format validation
■
▼
Architecture / tensor validation
■
▼
Checksum + metadata
■
▼
Controlled inference worker
■
▼
CatanIA game engine
■
▼
benchmark / Arena
```

The website owns the inference implementation. A submitted model supplies learned numerical parameters; the platform supplies the approved network architecture, observation encoder, legal-action masking, game engine and inference policy.

This is especially compatible with the current project because the README states that the structured network is the default network and that the observation is 2,503 floats with 325 discrete actions.

Requirement: do not accept an arbitrary architecture definition. Define a platform model format such as **CATANIA-1**, containing fixed architecture/version identifiers and compatible tensor shapes.

7. Model Format Requirements

Create a formal specification in both repositories. Example:

```
CATANIA-1
ruleset: catan-1v1-v1
observation_version: 1
observation_size: 2503
action_space_version: 1
action_count: 325
network: structured-v1
inference_modes:
- policy
- alphazero
serialization: approved PyTorch state_dict format
```

• Reject incompatible observation sizes.

• Reject incompatible action-space sizes.

• Reject missing or unexpected tensors.

• Reject NaN/Inf weights.

• Record the SHA-256 checksum of every accepted weight file.

• Record the exact model/rules/engine versions used for every Arena or user game.

• Never silently run a model from an old contract against a new encoder or action space.

• Keep historical model versions immutable.

8. Recommended Website Architecture

Start as a modular monolith plus background workers, not microservices. Keep clear internal modules and separate processes where workload isolation matters. This gives you a simple first deployment while preserving a path to scale.

9. Recommended Technology Stack

Layer	Recommendation	Reason
Frontend	React + TypeScript / Next.js	Structured component system, routing, typed API clients and strong ecosystem.
Backend API	Python + FastAPI	Natural fit with the existing Python engine and clean API/WebSocket support.
Game runtime	CatanIA Python package	Keeps one authoritative implementation of rules.
Database	PostgreSQL	Users, games, AI metadata, ELO, submissions and financial records.
Cache/queue	Redis	Background jobs, rate limiting and short-lived real-time state where useful.
Workers	Python worker processes	Reuse CatanIA inference/runtime and isolate expensive jobs.
Object storage	S3-compatible storage	Model weights, replay artifacts and larger files.
Reverse proxy	Caddy or Nginx	TLS termination and routing.
Containers	Docker + Docker Compose initially	Reproducible local/staging/production deployment.

Layer	Recommendation	Reason
CI/CD	GitHub Actions	Tests, builds, container images and deployment automation.
Monitoring	Structured logs + metrics + error tracking	Required to operate a public service reliably.

10. Frontend Architecture

```
frontend/
■■■ app/
■ ■■■ (public)/
■ ■ ■■■ page
■ ■ ■■■ leaderboard/
■ ■ ■■■ play/
■ ■ ■■■ create-ai/
■ ■ ■■■ submit-ai/
■ ■ ■■■ tutorials/
■ ■ ■■■ arena/
■ ■■■ account/
■ ■■■ ai/[id]/
■ ■■■ games/[id]/
■ ■■■ admin/
■■■ components/
■ ■■■ catan-board/
■ ■■■ ai-card/
■ ■■■ leaderboard/
■ ■■■ arena/
■ ■■■ forms/
■■■ lib/
■ ■■■ api/
■ ■■■ auth/
■ ■■■ websocket/
■■■ tests/
```

The frontend should be a presentation layer. Do not duplicate Catan rules in JavaScript. The current README specifically describes the browser as holding no rules, board generation or scoring. Preserve this design.

11. Required Web Pages

Page	Minimum requirements
Home	Explain the platform, current champion/Arena activity, CTA to play and create an AI.
Leaderboard	AI name, ELO, games, win rate, creator, profile image, ruleset/category filters and Arena explanation.
Play	Select AI, show access entitlement, create game, play in real time, reconnect, finish and display result.
AI Profile	Creator, description, version, ELO history, games, win rate, recent matches and play button.
Create AI	Explain local repository, setup, model contract, training workflow, local testing and how to submit weights.
Submit AI	Upload, validation progress, compatibility errors, benchmark status and final submission result.
My AIs	All submitted versions, status, performance, ELO, games and feedback.
Tutorials	Searchable/free content plus paid advanced content if monetization is enabled.
Arena TV	Live matches, board state, players, move history, match result and replay.
Account	Profile, authentication/security settings, games, AIs, purchases and creator earnings.
Admin	User moderation, AI moderation, model status, Arena controls, system health and audit logs.

These pages directly implement the Website Structure document's requested product areas: Leaderboard, Play against AI, Submit AI, Create AI, Tutorials, Arena TV and Account.

12. Backend Architecture

```
backend/
■■■ app/
■ ■■■ main.py
■ ■■■ config.py
■ ■■■ api/
■ ■ ■■■ auth.py
■ ■ ■■■ users.py
■ ■ ■■■ ais.py
■ ■ ■■■ games.py
■ ■ ■■■ leaderboard.py
■ ■ ■■■ submissions.py
```

```
■ ■ ■■■ arena.py
■ ■ ■■■ tutorials.py
■ ■ ■■■ payments.py
■ ■■■ domain/
■ ■ ■■■ games/
■ ■ ■■■ ais/
■ ■ ■■■ ratings/
■ ■ ■■■ entitlements/
■ ■■■ auth/
■ ■■■ db/
■ ■■■ services/
■ ■■■ realtime/
■ ■■■ integrations/
■■■ tests/
```

Use service/domain modules behind the HTTP endpoints. Avoid putting game logic, SQL and authentication logic directly inside route handlers.

13. Database Architecture

PostgreSQL is the system of record. At minimum, design these tables:

• **users** — account identity and profile.

• **sessions / refresh_tokens** — authentication state if your auth implementation uses them.

• **ais** — public AI identity and creator.

• **ai_versions** — immutable submitted model versions.

• **model_artifacts** — object-storage location, checksum and validation metadata.

• **games** — players, ruleset, engine version, model versions, seed, result and timestamps.

• **game_events / replays** — optional normalized events or a compact replay reference.

• **ratings** — ELO history rather than only the current number.

• **arena_matches** — scheduled/active/completed Arena matches.

• **submissions** — validation and review state.

• **tutorials** — content and publication status.

• **subscriptions / purchases** — entitlement records.

• **creator_revenue** — immutable accounting events.

• **audit_logs** — security/admin actions.

The existing local game recorder uses seed and move indices because the engine is deterministic. Preserve that idea online: store compact replay information plus enough version metadata to reproduce the game.

14. Game Runtime and Online Games

The web server must remain authoritative. The browser sends user intent; the backend validates and applies it using the CatanIA runtime.

```
Browser
■ action
▼
WebSocket/API
■
▼
Game Session
■
▼
CatanIA Runtime
■
■■■ legal_actions
■■■ apply
■■■ PublicView
```

```
■■■ state transition
■
▼
Persist result/event
■
▼
Broadcast new public state
```

This directly preserves the current project's one-source-of-truth philosophy and hidden-information boundary. The README states that the browser sends clicks while the server owns the rules and that the observation, web responses and game log are filtered consistently.

15. Authentication Requirements

Authentication is a production requirement, not a later convenience.

• Use HTTPS everywhere.

• Never store plaintext passwords.

• Use a mature password hashing algorithm such as Argon2id or an equivalent well-reviewed password hashing mechanism.

• Use secure, HttpOnly, SameSite cookies for browser sessions unless a different token architecture is deliberately chosen.

• Separate authentication from authorization.

• Implement email verification if email is required for the account model.

• Provide password reset with single-use, expiring tokens.

• Add optional 2FA later without changing the account model.

• Rate-limit login, password-reset and other abuse-prone endpoints.

• Invalidate sessions on explicit logout and allow users to revoke active sessions.

• Never put secrets in frontend code.

16. Authorization Model

Define roles and resource ownership explicitly:

```
anonymous
user
creator
moderator
admin
service/worker
```

• A user may edit only their own profile.

• A creator may manage only their own AI metadata and submissions.

• Only workers/services may transition validated models into Arena-ready state.

• Only admins/moderators may remove content or suspend accounts.

• Payments and revenue records must never be editable through ordinary user endpoints.

• Every privileged action should be auditable.

17. Security Requirements

Model upload security

• Accept only explicitly supported file formats.

• Limit file size before upload completion where possible.

• Upload into quarantine storage first.

• Validate archive/file structure if archives are supported.

• Do not execute, import or deserialize arbitrary executable Python objects from users.

• Prefer a safe, restricted state-dictionary representation over arbitrary PyTorch pickles.

• Verify tensor names, shapes, dtype, finite values and expected architecture version.

• Calculate a checksum and make accepted artifacts immutable.

• Run validation in a separate worker/container with no production database credentials.

• Do not give model validation workers unnecessary network access.

Web security

• TLS/HTTPS everywhere.

• Strict input validation using typed request schemas.

• Parameterized ORM/database queries.

• CSRF protection where cookie-based state-changing requests require it.

• Content Security Policy and secure HTTP headers.

• CORS restricted to the actual frontend origins.

• Rate limiting and abuse controls.

• No secrets committed to Git.

• Separate production credentials from development credentials.

• Regular dependency scanning and container image scanning.

• Audit logs for authentication, model submission, moderation and financial operations.

• Backups plus restore testing.

18. Docker Architecture

```
docker-compose.yml

services:
frontend
api
worker
scheduler
postgres
redis
reverse-proxy

Optional production-managed services:
object-storage
monitoring
error-tracking
```

Do not put everything into one container. At minimum, separate frontend, API and background workers. PostgreSQL and Redis can be containers in development and early staging; in production you may later replace them with managed services without changing application boundaries.

The worker should be the process that loads CatanIA and model weights. This prevents a slow AI match from blocking ordinary web requests.

19. Environment Configuration

```
APP_ENV=production
DATABASE_URL=...
REDIS_URL=...
OBJECT_STORAGE_BUCKET=...
OBJECT_STORAGE_ENDPOINT=...
OBJECT_STORAGE_ACCESS_KEY=...
```

```
OBJECT_STORAGE_SECRET_KEY=...
AUTH_SECRET=...
PAYMENT_SECRET=...
ALLOWED_ORIGINS=...
MODEL_FORMAT_VERSION=CATANIA-1
RULESET_VERSION=catan-1v1-v1
```

Use environment variables or a secret manager. Never commit real production secrets to either repository.

20. CI/CD Requirements

Every pull request to CatanIA-Web should run:

• Frontend lint/type checks.

• Backend lint/type checks.

• Unit tests.

• API/integration tests.

• Database migration checks.

• Security/dependency scanning.

• Docker build.

• A smoke test that starts the stack.

For CatanIA, preserve the existing large test suite. The README currently reports 937 tests and explicitly treats hidden-information leak tests as part of correctness.

21. How CatanIA Becomes a Dependency of the Web Repo

Do not copy the entire CatanIA repository into CatanIA-Web. Choose one of these approaches, in order of preference:

• **Preferred:** package the stable runtime portion of CatanIA and publish versioned releases internally/publicly. CatanIA-Web pins an exact version.

• **Alternative during early development:** use a Git dependency pinned to a commit/tag.

• **Avoid:** copying files manually between repositories.

```
CatanIA release
v0.1.0
■
▼
CatanIA-Web requirements
catania-runtime==0.1.0
```

The web platform can then upgrade deliberately rather than silently changing game behavior whenever the AI repository changes.

22. Recommended CatanIA Package Boundary

You do not necessarily need to reorganize the AI repository immediately. First identify the runtime subset that the website needs:

```
catan/
board
state
rules
topology
view
encoder
action_space
env
training/
model architecture
inference utilities
model loader

Not required by the web API:
self-play training
replay buffers
training chains
checkpoints
optimizer state
experiment dashboards
training workers
```

The current README explicitly separates the engine, interfaces and training responsibilities. The web deployment should make the same conceptual split at the package level.

23. AI Submission State Machine

```
DRAFT
■
▼
UPLOADED
■
▼
VALIDATING
■
■■■ invalid → REJECTED
■
▼
VALIDATED
■
▼
BENCHMARKING
■
■■■ failed → REJECTED / NEEDS_CHANGES
```

```
■
▼
ARENA_READY
■
▼
ACTIVE
■
■■■ superseded → ARCHIVED
```

This state machine should be represented in PostgreSQL, not inferred from filenames or object-storage folders.

24. Arena Architecture

```
Scheduler
■
▼
Match Queue (Redis)
■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
▼ ▼ ▼
Worker 1 Worker 2 Worker N
■ ■ ■
▼ ▼ ▼
CatanIA CatanIA CatanIA
+ model + model + model
■ ■ ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
▼
Match result
■
■■■■■■■■■■■■■■■
▼ ▼
ELO Replay
■ ■
■■■■■■■■■■■■■■■
▼
PostgreSQL
```

Reuse the existing Arena/match concepts rather than creating a second game implementation. The current repository already has an AlphaZero Arena that runs head-to-head matches across processes.

25. Realtime / WebSocket Requirements

• A player must be able to reconnect without corrupting the game.

• The server must authenticate the WebSocket connection.

• A player receives only their authorized public state.

• Never broadcast private opponent information.

• Use server-generated sequence numbers or equivalent ordering.

• Handle duplicate client messages idempotently where practical.

• Disconnecting a browser must not terminate the authoritative game immediately.

• Persist enough state/replay information to recover after a worker/API restart.

26. Game Persistence and Replay

The existing project records games using seed and move indices because deterministic replay reproduces the board, decks, rolls and observations.

For the website, store:

• game ID

• ruleset version

• engine version

• AI/model versions

• player IDs

• seed

• ordered actions/move indices

• result

• timestamps

• replay/checksum metadata

This gives you a compact, auditable foundation for Arena TV, user replays, debugging and dispute resolution.

27. Leaderboard Requirements

The Website Structure document explicitly calls for AI name, ELO, games played, win rate, creator name and profile picture, plus Arena explanation and the number of AIs in the Arena.

• Never calculate the leaderboard only in the browser.

• Store rating history.

• Associate each rating with a ruleset and rating season/version.

• Exclude invalid/unrated matches according to explicit rules.

• Keep historical ratings even if an AI is later archived.

• Show confidence/sample size information where useful so a 3-game AI is not visually equivalent to a 3,000-game AI.

28. Monetization Requirements

The supplied Website Structure proposes one free game per day initially, unlimited games for paying users, and creator revenue based on games played against their AI.

Treat this as an entitlement and accounting system:

```
Purchase / subscription
■
▼
Entitlement
■
▼
Can user start game?
■
▼
Game usage event
■
▼
Creator revenue event
■
▼
Immutable accounting ledger
```

Do not make payment status a frontend variable. The backend must decide whether a user is entitled to start a game.

29. Scalability Strategy

Design for horizontal scaling without deploying a complex microservice architecture on day one.

```
Stage 1
1 VPS
■■■ reverse proxy
■■■ frontend
■■■ API
■■■ worker
■■■ PostgreSQL
```

```
■■■ Redis

Stage 2
Managed PostgreSQL
Managed object storage
Separate worker server
CDN
API replicas

Stage 3
Load balancer
Multiple API replicas
Multiple game workers
Multiple Arena workers
Managed Redis
Dedicated model/inference workers
Separate staging/production clusters
```

The key is to keep APIs stateless where possible and move long-running/CPU-heavy work into queues. Then scaling the number of workers does not require changing the website architecture.

30. Observability Requirements

• Centralized structured logs.

• Request IDs/correlation IDs.

• API latency and error-rate metrics.

• Active game count.

• WebSocket connection count.

• Arena queue depth.

• Arena match duration.

• Model validation failure rate.

• AI inference latency.

• Database health.

• Worker health.

• Authentication failure rate.

• Storage usage.

• Alerts for service failures and abnormal resource usage.

31. Backup and Disaster Recovery

• Automated PostgreSQL backups.

• Backups stored outside the main server.

• Object-storage versioning/retention for model artifacts.

• Documented restore procedure.

• Regular restore tests.

• Keep Git repositories as the source of code truth.

• Keep model artifacts immutable and addressable by checksum/version.

32. Repository Migration Plan

Step	Work	Definition of done
Step 1	Freeze a known-good CatanIA commit.	Create a tag such as the first production baseline. Run the full test suite.
Step 2	Document the runtime contract.	Ruleset, engine version, observation size, action count, model architecture and serialization format.
Step 3	Create CatanIA-Web.	Add README, architecture docs, issue templates, CI and basic Docker skeleton.
Step 4	Package the CatanIA runtime.	Make the web repository able to install a pinned CatanIA release/commit.
Step 5	Build the local web stack.	Frontend + FastAPI + PostgreSQL + Redis + worker in Docker Compose.
Step 6	Recreate the current local game online.	One user can log in and play against an existing official AI.
Step 7	Add accounts and persistence.	Users, games, AI profiles and replay metadata.
Step 8	Add model submission.	Weight upload, validation, checksum, model registry and status machine.
Step 9	Add online AI inference.	A submitted compatible model can play a controlled game.
Step 10	Add leaderboard/ELO.	Record completed matches and rating history.
Step 11	Add Arena workers.	Automated AI-vs-AI matches and replay.
Step 12	Add Arena TV.	Live/replay frontend.
Step 13	Add tutorials and creator dashboard.	Content, performance and submission feedback.
Step 14	Add payments.	Only after the free gameplay loop is reliable.
Step 15	Deploy production.	Domain, TLS, backups, monitoring, rate limits and operational runbooks.

33. Minimum Viable Public Launch

Do **not** wait until every planned feature is complete. The first public release should prove the central platform loop:

```
Create account
↓
Choose AI
↓
Play Catan online
↓
Game finishes
↓
Result saved
↓
AI statistics update
↓
User can play again
```

Then add the differentiating community loop:

```
Create AI locally
↓
Train locally
↓
Submit weights
↓
Validation
↓
Arena evaluation
↓
ELO
↓
Public AI profile
↓
Other users play against it
```

The supplied website concept already centers the platform around playing against AI, submitting AI, creating AI, tutorials, Arena TV and accounts.

34. Go-Live Checklist

■ CatanIA has a tagged production baseline.

■ CatanIA-Web has its own Git repository.

■ CatanIA-Web pins an exact CatanIA runtime version.

■ The model contract is documented.

■ Ruleset and engine versions are recorded with games.

■ Docker Compose starts the complete development stack.

■ CI passes on pull requests.

■ Production secrets are outside Git.

■ HTTPS is enabled.

■ Authentication is implemented securely.

■ Authorization/ownership checks are tested.

■ PostgreSQL migrations are automated.

■ Backups are automated and restore-tested.

■ Model uploads are quarantined and validated.

■ User code is never executed.

■ Accepted weights are checksummed and immutable.

■ Model workers have restricted permissions.

■ Online game state is server-authoritative.

■ WebSocket reconnect works.

■ Private information cannot cross the player boundary.

■ Leaderboard is backed by database records.

■ Arena workers are isolated from the API process.

■ Rate limiting is active.

■ Logs and monitoring are active.

■ Error alerts are configured.

■ A rollback procedure exists.

■ Terms/privacy/security policies are ready before public launch.

35. Definition of Done for the Architecture

The architecture is successful when the following statements are true:

• A person can clone CatanIA, install the documented dependencies and train/play locally without needing the website repository.

• A person can clone CatanIA-Web and run the website stack independently.

• CatanIA-Web never contains a second copy of the Catan rules.

• A CatanIA version can be pinned, upgraded and rolled back deliberately.

• A user can submit weights without submitting executable code.

• The platform can validate a model without trusting the uploader.

• The same CatanIA runtime powers local play, online play and Arena matches.

• A model/game result can always be traced to exact versions and a checksum.

• API servers can scale horizontally while workers scale independently.

• A database failure or worker restart cannot silently corrupt a completed game.

• The platform can add new pages/features without rewriting the game engine.

• Payments, creator revenue and moderation are separate from the core game rules.

36. First Sprint — What I Would Implement Now

Do not begin with payments, Arena TV or a sophisticated cloud architecture. The first sprint should establish the repository boundary and prove that the current game can be consumed by the new website.

• Create the CatanIA-Web repository.

• Create the Docker Compose development stack.

• Create a FastAPI backend.

• Create a React/TypeScript frontend.

• Create PostgreSQL migrations.

• Create a minimal CatanIA runtime package/release.

• Install that package from CatanIA-Web.

• Implement a health endpoint proving the runtime is available.

• Implement user registration/login.

• Implement an AI registry containing the current official champion.

• Implement one online game against that champion.

• Implement WebSocket state updates.

• Persist the completed game with ruleset/engine/model metadata.

• Write the first integration tests for hidden-information isolation.

• Deploy this minimal stack to staging before adding submissions.

This first sprint gives you the most important proof: **the two repositories work together without merging their responsibilities.**

37. Final Architecture Summary

```
CatanIA
■
■■■ authoritative Catan rules
■■■ observation / action space
■■■ AI architectures
■■■ training
■■■ inference
■■■ local play
■■■ research / benchmarking

versioned runtime contract
■
▼

CatanIA-Web
■
■■■ Frontend
■ ■■■ Home
■ ■■■ Leaderboard
■ ■■■ Play
■ ■■■ Create AI
■ ■■■ Submit AI
■ ■■■ Tutorials
■ ■■■ Arena TV
■ ■■■ AI profiles
■ ■■■ Account
■
■■■ Backend
■ ■■■ Authentication
■ ■■■ Users
■ ■■■ Games
■ ■■■ AI registry
■ ■■■ Weight validation
■ ■■■ Leaderboard
■ ■■■ Arena
■ ■■■ Payments
■ ■■■ Admin
■
■■■ Infrastructure
■ ■■■ Docker
■ ■■■ PostgreSQL
■ ■■■ Redis
■ ■■■ Object storage
■ ■■■ Reverse proxy / TLS
■ ■■■ CI/CD
■
■■■ Workers
■■■ Model validation
■■■ Online inference
■■■ Arena matches
■■■ Rating / replay processing
```

Core principle: CatanIA is the engine and laboratory. CatanIA-Web is the product and platform. The versioned model/runtime contract is the bridge between them.

Source notes: The architecture above is based on the current CatanIA README and the supplied Website Structure document. Where the source documents do not specify an implementation technology, the recommendations in this document are architectural proposals rather than existing project facts.